News & Context Explainer

An LLM Agent Built from Scratch

Yue Wu

The Problem

You read a headline.

"US imposes 145% tariffs on Chinese goods"

But you don't know:

- What triggered this?
- How did US-China trade get here?
- What does 145% actually cost in dollars?

You open 5 tabs, skim Wikipedia, do some math...

20 minutes later, you kind of get it.

The Gap

Headline $\longrightarrow$ Understanding

requires: current news
+ historical context
+ numbers in perspective

Why Not Just Ask ChatGPT?

Plain LLM

- × **Hallucination** — confidently states wrong facts with no source
- × **Knowledge cutoff** — unaware of anything after training date
- × **No calculation** — makes arithmetic errors on multi-step math
- × **No citations** — no way to verify where the answer came from
- × **Static** — same answer every time, regardless of current events

LLM Agent (ours)

- ✓ **Grounded** — answers backed by real web pages & Wikipedia
- ✓ **Up-to-date** — searches the web for today's information
- ✓ **Exact math** — delegates numbers to a safe calculator tool
- ✓ **Cited** — every fact has a traceable source URL
- ✓ **Adaptive** — plans which tools to use based on the question

The Solution

News & Context Explainer

An autonomous LLM agent that bridges the gap

Current News

What's happening right now

Historical Context

How we got here, key players

Numbers in Perspective

What the figures actually mean

One question in. One complete answer out.

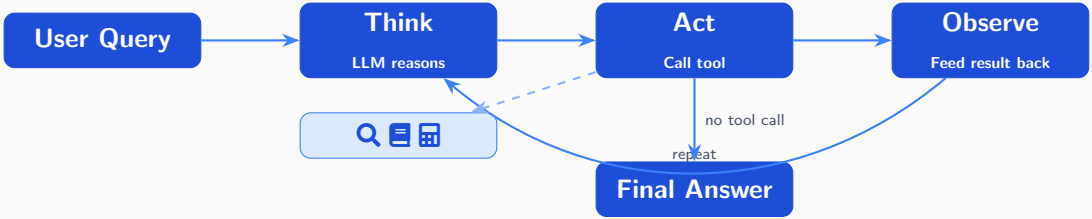
Three Tools, One Purpose

Tool	Source	Role
`web_search`	DuckDuckGo	Latest headlines & current facts
`wikipedia_lookup`	Wikipedia REST API	Historical background & context
`calculator`	Python AST (safe eval)	Tariffs, GDP ratios, percentages

Example — US-China Trade War:

1. `web_search("US China tariffs 2025")` → latest headlines
2. `wikipedia_lookup("China{United States trade war")` → history
3. `calculator("500000000000 * 0.145")` → $72.5B cost

Agent Architecture — ReAct Loop



- Written **from scratch** — no LangChain, no CrewAI
- Max 10 iterations (loop guard)
- **LLM:** Qwen3-32B via Groq (free tier)
- Tool schemas in OpenAI JSON format

Key Design Choices

Decision	Rationale
Groq + Qwen3-32B	Free tier, fast inference, strong tool calling
OpenAI-compatible API	Keeps tool schemas provider-agnostic
DuckDuckGo search	No API key required, no rate-limit costs
Wikipedia REST API	Structured, reliable, always available
Safe AST calculator	No `eval()` security risk
Gradio 6 UI	Minimal setup, collapsible tool traces

User Interface

Built with **Gradio 6**:

- Chat-style bubble layout
- Example news prompts to click
- **Collapsible tool traces**
  show reasoning steps
- Clear conversation button
- Runs locally at `localhost:7860`

[Live demo here]

News & Context Explainer

User: What's happening with the US-China trade war?

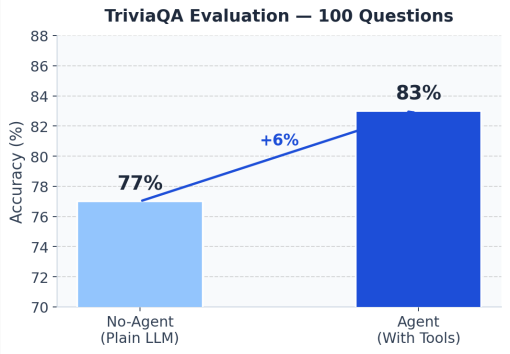
▷ web_search(...)
▷ wikipedia_lookup(...)
▷ calculator(...)

Agent: The US-China trade war...

Evaluation — Agent vs. Plain LLM

Setup

- Dataset: **TriviaQA** (public, HuggingFace)
- 100 validation questions
- Each question run **twice**: with tools vs. without tools
- Scoring: answer-in-prediction (standard TriviaQA metric)



Result: +6% improvement

Avg 1.0 tool calls/question

0 crashes across 100 runs

Summary

What was built:

- ReAct agent loop from scratch
- 3 tools: search, Wikipedia, calculator
- Gradio chat UI with tool traces
- Evaluation on TriviaQA (100 Qs)

Key result:

Agent outperforms bare LLM by +6% on a real public benchmark

GitHub:

`github.com/202520030411/LLM_agent_from_scratch`

Stack:

Python · Groq · Qwen3-32B
Gradio · DuckDuckGo · Wikipedia